# Programming Assignment 3 Report

Lizi Luo (lluo040) 862605965

## Program 1 Multiset (BST)

**File:** `multiset.h` `Multiset()`

Creates an empty BST multiset (`root = nullptr`, `size = 0`).

`Insert(const K& key)`

`Insert` follows BST order.

1. Search key position.
2. If key exists, increment `count`.
3. If key does not exist, create node.

Then it increase total size by 1.

`Remove(const K& key)`

`Remove` has duplicate-aware logic.

1. If key not found: throw `"Invalid key"`.
2. If `count > 1`: decrement count only.
3. If `count == 1`: remove BST node.

For a 2-child node, replacement key is the inorder sucessor (minimum in right subtree).

### Search/API functions

`Contains(key)` and `Count(key)` use normal BST search.
`Min()` and `Max()` go to leftmost/rightmost node.
`Floor(key)` and `Ceil(key)` keeps the best candidate during traversal.

Core rule:

$$\text{Floor}(x) = \max\{k \mid k \le x\}, \qquad \text{Ceil}(x) = \min\{k \mid k \ge x\}$$

If operation require a value and set is empty, throw `"Empty multiset"`.

Complexity:

$$T(N) = O(h) \approx O(\log N)$$

where $h$ is tree hight (worst case $O(N)$ for non-balanced BST).

### Tester updates (`test_multiset.cc`)

I added tests beyond the simple starter file: - empty state and exception behavior, - duplicate insert/remove, - full key deletion after repeated remove, - `Floor`/`Ceil` exact, in-between, and no-match, - remove node with children, - mixed tree operations.

Test goal:

$$\text{basic behavior} \rightarrow \text{duplicate correctness} \rightarrow \text{edge cases}$$

## Program 2 Prime Factors CLI

**File: `prime_factors.cc` `main()`**

`main` does:

1. Parse arguments.
2. Validate number and comand.
3. Compute factors.
4. Run one command (`all|min|max|near`).

**`ComputePrimeFactors(unsigned int n, Multiset<int>& factors)`**

Uses trial division:

1. Divide by 2 while possible.
2. Divide by odd numbers from 3 to $\sqrt{n}$.
3. If remaining $n > 1$, insert it as final prime.

Each prime factor are inserted into `Multiset<int>` (duplicates stored by count).

### Output commands

`PrintAll`: print all factors in ascending order with multiplicty.
`PrintMin` / `PrintMax`: print one endpoint factor with count.
`PrintNear`: - `near p` exact factor, - `near +p` nearest greater factor, - `near -p` nearest smaller factor.

No result -> `No match`.
No factors -> `No prime factors`.

### Complexity

Factorization:

$$T(n) = O(\sqrt{n})$$

Command query on multiset:

$$O(h) \approx O(\log M)$$

where $M$ is number of stored factor entries.